

EDA_2

August 9, 2026

```
[1]: """
Initialization & Configuration Cell
-----

Purpose:
    Establish the runtime environment, import required libraries, and define
    configuration parameters for the Geolife EDA workflow. This cell sets up
    directory paths, file limits, and output locations used throughout the
    notebook.

Overview:
    - Imports core Python libraries for filesystem access, numerical operations,
      data manipulation, and visualization.
    - Defines constants controlling dataset location, output directories, and
      performance-related limits (e.g., number of files to process, maximum
      points per trajectory).
    - Ensures the output directory exists so downstream visualization cells
      can save figures without raising errors.

Inputs:
    - Local filesystem paths pointing to the Geolife dataset.
    - User-defined configuration parameters.

Outputs:
    - Global constants available to all subsequent cells.
    - Created output directory (`eda_figures/`) if not already present.

Notes:
    - Adjust MAX_FILES and MAX_POINTS_PER_FILE depending on hardware capacity
      and desired EDA depth. Lower values improve speed; higher values improve
      completeness.
    - This cell performs no data loading; it only prepares the environment.
    """

# -----
# Library Imports
# -----
import os                      # Filesystem operations (directory creation, path handling)
```

```

import glob          # Pattern-based file discovery (used for GPX/PLT file
    ↪enumeration)
import numpy as np    # Numerical operations and array handling
import pandas as pd   # Tabular data manipulation and parquet loading
import matplotlib.pyplot as plt # Visualization backend for EDA plots

# -----
# Configuration Parameters
# -----
# Root directory containing the Geolife dataset. Using raw string literal to
# avoid escaping backslashes on Windows paths.
DATA_ROOT = r"C:\Users\gb630\OneDrive\USD AAI\USD AAI\AAI-590 CAPSTONE\FINAL
    ↪PROJECT\Data"

# Full path to the preprocessed parquet file. Parquet is preferred for large
# trajectory datasets due to efficient columnar storage and fast I/O.
PARQUET_PATH = f"{DATA_ROOT}\geolife.parquet"

# Directory where all EDA figures will be saved. Keeping this separate ensures
# reproducibility and avoids cluttering the notebook directory.
OUTPUT_DIR = "eda_figures"

# Performance tuning parameters:
MAX_FILES = 300          # Upper bound on number of trajectory files to load.
                        # Increase for full dataset analysis; decrease for
    ↪speed.
MAX_POINTS_PER_FILE = 5000 # Limit number of GPS points per file to prevent
                        # memory pressure and long render times for large
    ↪tracks.

# -----
# Directory Initialization
# -----
# Create the output directory if it does not already exist. `exist_ok=True`
# prevents errors when rerunning the notebook.
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

```

[2]: def load_geolife_plt(file_path):
    """
    Load a single GeoLife .plt trajectory file.

    Purpose:
        Parse a raw GeoLife trajectory file into a clean, time-ordered
        pandas DataFrame suitable for downstream EDA and modeling.

    GeoLife Format Details:
        Each .plt file contains 6 header rows followed by GPS records with:
    """

```

lat, lon, unused, altitude, days, date, time

- *'unused' is a placeholder column in the GeoLife format and is intentionally ignored.*
- *'days' is a fractional day count used internally by GeoLife but is not required for typical EDA workflows.*
- *'date' and 'time' must be combined into a proper datetime object.*

Inputs:

file_path (str): Full filesystem path to a .plt file.

Outputs:

df (DataFrame): Cleaned trajectory data with columns:

*['lat', 'lon', 'unused', 'altitude', 'days', 'date', 'time',
↳ 'datetime']*

Notes:

- *Rows with invalid or missing datetime/lat/lon are removed.*
- *Trajectories are truncated to MAX_POINTS_PER_FILE to prevent memory pressure and long render times.*
- *Sorting by datetime ensures correct temporal ordering.*

"""

Read the .plt file, skipping the 6 non-data header rows.

Explicit column names ensure consistent schema across all files.

```
df = pd.read_csv(  
    file_path,  
    skiprows=6,  
    header=None,  
    names=["lat", "lon", "unused", "altitude", "days", "date", "time"]  
)
```

Combine date + time into a proper datetime object.

*# Using errors="coerce" ensures malformed entries become NaT instead of
↳ crashing.*

```
df["datetime"] = pd.to_datetime(  
    df["date"].astype(str) + " " + df["time"].astype(str),  
    errors="coerce"  
)
```

Drop rows missing essential fields (datetime, lat, lon).

This prevents downstream geospatial operations from failing.

```
df = df.dropna(subset=["datetime", "lat", "lon"])
```

Sort by timestamp to ensure trajectory is in chronological order.

```
df = df.sort_values("datetime").reset_index(drop=True)
```

```

# Truncate long trajectories to improve performance during EDA.
# MAX_POINTS_PER_FILE is defined globally in the configuration cell.
if len(df) > MAX_POINTS_PER_FILE:
    df = df.iloc[:MAX_POINTS_PER_FILE].copy()

return df

def load_sample_trajectories(data_root, max_files=300):
    """
    Load a sample of GeoLife trajectory files from the dataset.

    Purpose:
        Recursively search the GeoLife directory structure for .plt files,
        load each trajectory using `load_geolife_plt`, and return a list
        of cleaned DataFrames.

    Directory Structure Assumption:
        GeoLife dataset is organized as:
            <root>/<user_id>/Trajectory/*.plt

        The recursive glob pattern ensures all user folders are scanned.

    Inputs:
        data_root (str): Root directory containing GeoLife user folders.
        max_files (int): Maximum number of .plt files to load. Useful for
            limiting EDA runtime on large datasets.

    Outputs:
        trajectories (list[DataFrame]): A list of cleaned trajectory DataFrames.
            Each DataFrame includes a 'source_file'
            column identifying its origin.

    Notes:
        - Files with fewer than 10 valid points are skipped to avoid noise.
        - Exceptions are caught and logged so a single corrupt file does not
            interrupt the entire loading process.
        - The function prints a summary count of successfully loaded_
        ↪trajectories.
    """

    # Recursively search for all .plt files under the GeoLife directory.
    # The pattern "**/Trajectory/*.plt" matches all user folders.
    plt_files = glob.glob(
        os.path.join(data_root, "**", "Trajectory", "*.plt"),
        recursive=True
    )

```

```

# Limit the number of files processed for performance reasons.
plt_files = plt_files[:max_files]

trajectories = []

# Iterate through each file and attempt to load it.
for file_path in plt_files:
    try:
        df = load_geolife_plt(file_path)

        # Skip extremely short trajectories (likely noise or corrupt files).
        if len(df) > 10:
            df["source_file"] = file_path # Track provenance for debugging/
↳ EDA.
            trajectories.append(df)

    except Exception as e:
        # Defensive programming: log and skip problematic files.
        print(f"Skipping {file_path}: {e}")

print(f"Loaded {len(trajectories)} trajectories.")
return trajectories

```

```

[3]: def haversine_meters(lat1, lon1, lat2, lon2):
    """
    Compute great-circle distance between two GPS coordinates using the
    ↳ Haversine formula.

    Purpose:
    Convert latitude/longitude pairs into a geodesic distance (in meters)
    assuming a spherical Earth. This is the standard method for computing
    distances between GPS points in trajectory datasets such as GeoLife.

    Inputs:
    lat1, lon1 : float or array-like
        Latitude/longitude of the first point in degrees.
    lat2, lon2 : float or array-like
        Latitude/longitude of the second point in degrees.

    Outputs:
    distance_meters : float or array-like
        Great-circle distance between the two points in meters.

    Notes:
    - Earth radius is approximated as 6,371,000 meters.
    - Inputs are converted to radians before computation.
    """

```

```

        - Supports vectorized NumPy operations for efficient batch processing.
    """

    R = 6371000 # Earth radius in meters (standard geodesic approximation)

    # Convert degrees → radians for trigonometric operations.
    lat1 = np.radians(lat1)
    lon1 = np.radians(lon1)
    lat2 = np.radians(lat2)
    lon2 = np.radians(lon2)

    # Compute deltas.
    dlat = lat2 - lat1
    dlon = lon2 - lon1

    # Haversine formula components.
    a = (
        np.sin(dlat / 2)**2
        + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2
    )
    c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))

    return R * c

def bearing_radians(lat1, lon1, lat2, lon2):
    """
    Compute the bearing (heading) from point A → point B in radians.

    Purpose:
        Determine the direction of travel between two GPS coordinates.
        Bearing is essential for motion analysis, turn-rate estimation,
        and trajectory classification.

    Inputs:
        lat1, lon1 : float or array-like
                     Starting coordinate in degrees.
        lat2, lon2 : float or array-like
                     Ending coordinate in degrees.

    Outputs:
        bearing : float or array-like
                 Direction of travel in radians, measured clockwise from north.

    Notes:
        - Bearing is undefined when points are identical; downstream logic
          should handle NaN values appropriately.
    """

```

```

        - Uses the standard geodesic bearing formula.
    """

    # Convert degrees → radians.
    lat1 = np.radians(lat1)
    lat2 = np.radians(lat2)
    dlon = np.radians(lon2 - lon1)

    # Compute bearing components.
    x = np.sin(dlon) * np.cos(lat2)
    y = (
        np.cos(lat1) * np.sin(lat2)
        - np.sin(lat1) * np.cos(lat2) * np.cos(dlon)
    )

    return np.arctan2(x, y)

def add_motion_features(df):
    """
    Add motion-derived features to a GeoLife trajectory DataFrame.

    Purpose:
    Enhance raw GPS trajectories with physically meaningful features
    such as speed, acceleration, heading, turn rate, and distance traveled.
    These features are essential for movement classification, anomaly
    detection, and transportation-mode inference.

    Inputs:
    df (DataFrame):
        Must contain columns ['lat', 'lon', 'datetime'] sorted in
        chronological order.

    Outputs:
    df_out (DataFrame):
        Copy of the input DataFrame with additional columns:
        - dt : time delta between points (seconds)
        - lat_prev, lon_prev : previous GPS coordinate
        - distance_m : haversine distance between points
        - speed_mps : meters per second
        - heading : bearing in radians
        - acceleration_mps2 : change in speed / dt
        - heading_change : raw change in heading
        - heading_change_wrapped : normalized heading change (- to )
        - turn_rate_radps : heading change per second

    Notes:
    """

```

```

- Infinite values (e.g., division by zero) are replaced with NaN.
- Rows with missing motion features are dropped to ensure clean
  downstream modeling.
- This function assumes the DataFrame is already sorted by datetime.
"""

df = df.copy() # Avoid mutating original trajectory.

# Time delta between consecutive points (seconds).
df["dt"] = df["datetime"].diff().dt.total_seconds()

# Previous GPS coordinates (used for distance and bearing calculations).
df["lat_prev"] = df["lat"].shift(1)
df["lon_prev"] = df["lon"].shift(1)

# Great-circle distance between consecutive points.
df["distance_m"] = haversine_meters(
    df["lat_prev"], df["lon_prev"], df["lat"], df["lon"]
)

# Speed (meters per second). Division by zero handled later.
df["speed_mps"] = df["distance_m"] / df["dt"]

# Heading (bearing) between consecutive points.
df["heading"] = bearing_radians(
    df["lat_prev"], df["lon_prev"], df["lat"], df["lon"]
)

# Acceleration: change in speed divided by time delta.
df["acceleration_mps2"] = df["speed_mps"].diff() / df["dt"]

# Raw heading change (radians).
df["heading_change"] = df["heading"].diff()

# Normalize heading change to range [-, ] to avoid wrap-around artifacts.
df["heading_change_wrapped"] = ((df["heading_change"] + np.pi) % (2*np.pi))
↪- np.pi

# Turn rate: normalized heading change per second.
df["turn_rate_radps"] = df["heading_change_wrapped"] / df["dt"]

# Replace infinite values (e.g., division by zero) with NaN.
df = df.replace([np.inf, -np.inf], np.nan)

# Drop rows missing essential motion features.
df = df.dropna(subset=["dt", "speed_mps", "acceleration_mps2",
↪ "turn_rate_radps"])

```



```
return df
```

```
[4]: """
Trajectory Processing & EDA Filtering Cell
-----
Purpose:
    Load a sample of GeoLife trajectories, compute motion-derived features
    (speed, acceleration, turn rate, etc.), concatenate all processed
    trajectories into a single DataFrame, and apply EDA-appropriate
    filtering constraints to remove unrealistic or noisy GPS artifacts.

Workflow:
    1. Load raw .plt trajectories using `load_sample_trajectories`.
    2. Apply motion-feature engineering via `add_motion_features`.
    3. Concatenate all processed trajectories into one DataFrame.
    4. Filter out physically impossible or noisy motion values.
    5. Produce a clean dataset (`eda_data`) for visualization and analysis.

Outputs:
    - eda_data (DataFrame): Cleaned, filtered motion dataset suitable for EDA.
    - Printed shape of the final dataset.

Notes:
    - Filtering thresholds (speed, acceleration, turn rate) are chosen to
      remove GPS jitter, sensor noise, and unrealistic motion artifacts.
    - Rows with dt <= 0 are removed because negative or zero time deltas
      indicate timestamp irregularities or duplicate points.
    - This cell assumes that all upstream functions have already validated
      trajectory structure and datetime ordering.
"""

# -----
# 1. Load raw trajectories from disk
# -----
trajectories = load_sample_trajectories(DATA_ROOT, MAX_FILES)

processed = []

# -----
# 2. Compute motion features for each trajectory
# -----
for traj in trajectories:
    feat = add_motion_features(traj)

    # Skip extremely short trajectories; they provide little analytical value.
    if len(feat) > 10:
```

```

        processed.append(feats)

# -----
# 3. Concatenate all processed trajectories
# -----
all_data = pd.concat(processed, ignore_index=True)

# -----
# 4. Apply EDA filtering constraints
# -----
# These constraints remove:
# - dt <= 0: invalid or duplicate timestamps
# - dt > 60: GPS gaps too large for reliable motion inference
# - speed_mps < 0 or > 60: unrealistic speeds (60 m/s 134 mph)
# - acceleration_mps2 > 20: extreme spikes likely caused by GPS jitter
# - turn_rate_radps > 5: abrupt heading changes not physically plausible
eda_data = all_data[
    (all_data["dt"] > 0)
    & (all_data["dt"] <= 60)
    & (all_data["speed_mps"] >= 0)
    & (all_data["speed_mps"] <= 60)
    & (all_data["acceleration_mps2"].abs() <= 20)
    & (all_data["turn_rate_radps"].abs() <= 5)
].copy()

# -----
# 5. Summary output
# -----
print("Processed data shape:", eda_data.shape)

```

Loaded 296 trajectories.

Processed data shape: (364187, 19)

```

[5]: """
Timestamp Interval Distribution Plot
-----
Purpose:
    Visualize the distribution of time differences (`dt`) between consecutive
    GPS points in the cleaned EDA dataset. This histogram helps identify
    sampling frequency characteristics, detect irregular timestamp gaps,
    and validate that filtering thresholds (e.g., dt <= 60 seconds) are
    producing a realistic motion dataset.

Inputs:
    - eda_data (DataFrame): Must contain a numeric 'dt' column representing
      time deltas in seconds.

```

Outputs:

- Histogram figure displayed inline.
- No return value; figure is rendered directly in the notebook.

Notes:

- Using 60 bins provides 1-second resolution across the 0-60 second range.
- Color palette follows a professional, muted tone suitable for reports.
- ``tight_layout()`` ensures labels and titles are not clipped when exporting.

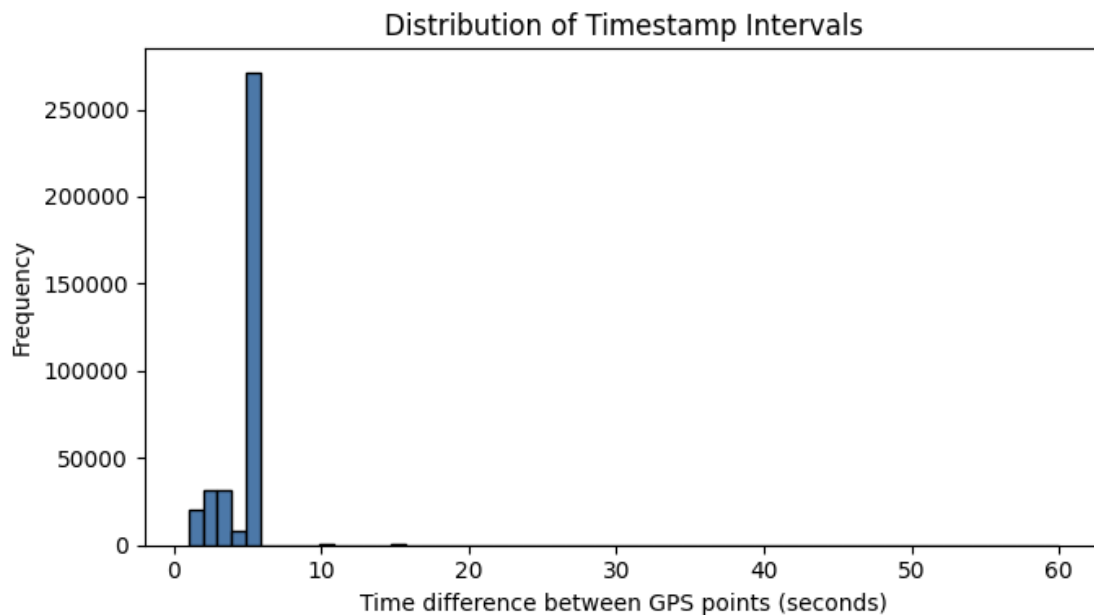
"""

```
plt.figure(figsize=(7, 4)) # Set figure size for readability in reports.

# Histogram of timestamp intervals.
# This reveals how frequently GPS points are recorded and highlights gaps.
plt.hist(
    eda_data["dt"],
    bins=60,
    color="#4C78A8",
    edgecolor="black"
)

# Axis labels and title for clarity in presentations and APA reports.
plt.xlabel("Time difference between GPS points (seconds)")
plt.ylabel("Frequency")
plt.title("Distribution of Timestamp Intervals")

plt.tight_layout() # Prevent label clipping when saving/exporting.
plt.show() # Render the figure inline.
```



```
[6]: """
Representative Trajectory Speed Profile Visualization
-----

Purpose:
    Select a representative trajectory from the processed dataset, apply
    EDA-appropriate filtering constraints, compute elapsed time relative to
    the start of the trajectory, and visualize the speed profile over time.

Workflow:
    1. Identify the longest processed trajectory (proxy for completeness).
    2. Apply motion-quality filters (dt, speed) to remove noisy segments.
    3. Compute elapsed time in seconds from the first timestamp.
    4. Plot speed (m/s) as a function of elapsed time.

Why This Matters:
    - A representative speed profile helps validate motion-feature extraction.
    - Reveals stop-and-go behavior, acceleration patterns, and travel mode.
    - Useful for diagnosing GPS jitter, timestamp irregularities, or
      unrealistic speed spikes.

Inputs:
    - processed (list[DataFrame]): List of motion-enhanced trajectories.

Outputs:
    - Inline plot of speed vs. elapsed time.
    - No returned value; visualization is rendered directly.

Notes:
    - Filtering thresholds match earlier EDA constraints for consistency.
    - The representative trajectory is chosen using `max(processed, key=len)`
      to ensure sufficient temporal coverage.
    """

# -----
# 1. Select a representative trajectory (longest = most data)
# -----
representative = max(processed, key=len).copy()

# -----
# 2. Apply motion-quality filters to remove noisy segments
# -----
representative = representative[
    (representative["dt"] > 0)          # Remove invalid or duplicate timestamps
    & (representative["dt"] <= 60)      # Remove large gaps that distort speed

```

```

    & (representative["speed_mps"] >= 0)
    & (representative["speed_mps"] <= 60) # Cap at ~134 mph to remove GPS
↳ spikes
].copy()

# -----
# 3. Compute elapsed time from the start of the trajectory
# -----
representative["elapsed_seconds"] = (
    representative["datetime"] - representative["datetime"].iloc[0]
).dt.total_seconds()

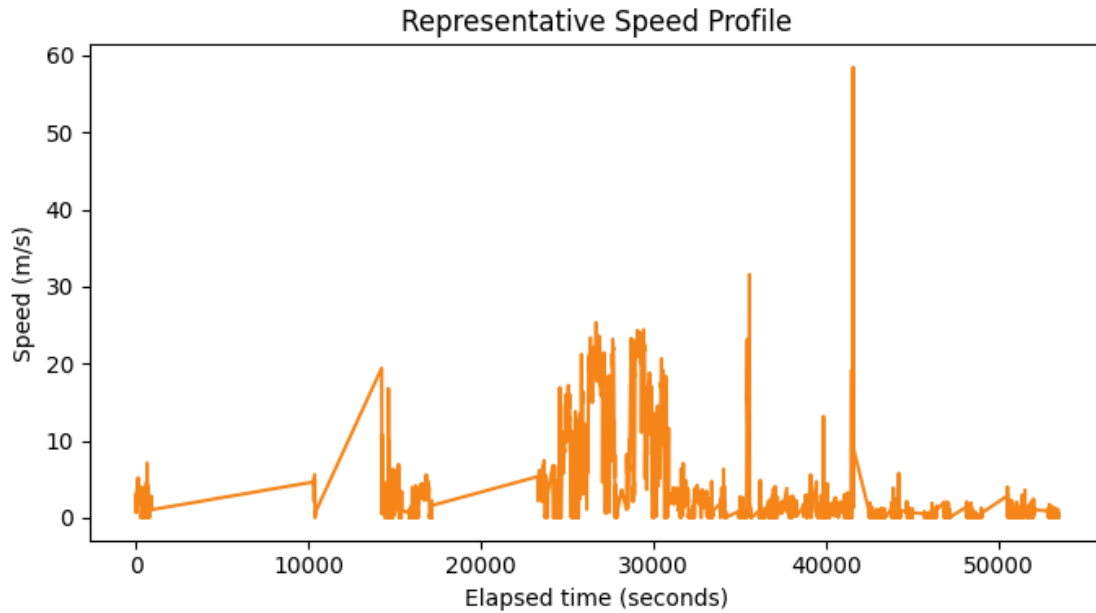
# -----
# 4. Plot speed profile over elapsed time
# -----
plt.figure(figsize=(7, 4))

plt.plot(
    representative["elapsed_seconds"],
    representative["speed_mps"],
    color="#F58518", # Muted orange for visual contrast
    linewidth=1.5    # Slightly thicker line for readability in reports
)

plt.xlabel("Elapsed time (seconds)")
plt.ylabel("Speed (m/s)")
plt.title("Representative Speed Profile")

plt.tight_layout() # Prevent clipping of labels when exporting
plt.show()

```



```
[7]: """
Acceleration & Turn-Rate Distribution Visualization
-----
Purpose:
    Generate side-by-side histograms showing the distribution of two key
    motion-derived features:
        1. Acceleration (m/s2)
        2. Turn rate (rad/s)

    These distributions help validate the physical realism of the cleaned
    GeoLife dataset and reveal whether GPS noise, jitter, or preprocessing
    thresholds are producing plausible motion characteristics.

Why These Features Matter:
    - Acceleration identifies stop-and-go behavior, sudden braking, and
      potential transportation mode transitions.
    - Turn rate highlights directional changes, sharp turns, and trajectory
      smoothness-important for mode classification and anomaly detection.

Inputs:
    - eda_data (DataFrame): Must contain numeric columns:
      ['acceleration_mps2', 'turn_rate_radps']

Outputs:
    - Inline figure containing two histograms.
    - No return value; visualization is rendered directly.
```

Notes:

- 80 bins provide fine-grained resolution for both distributions.
- Colors follow a professional palette suitable for APA-formatted reports.
- ``tight_layout()`` ensures proper spacing when exporting figures.

"""

Create a 1x2 subplot layout for side-by-side comparison.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
# -----
```

```
# Left Plot: Acceleration Distribution
```

```
# -----
```

```
axes[0].hist(
```

```
    eda_data["acceleration_mps2"],
```

```
    bins=80,                    # Fine resolution for subtle acceleration patterns.
```

```
    color="#54A24B",           # Muted green for professional visual tone.
```

```
    edgecolor="black"          # Clear bin boundaries for readability.
```

```
)
```

```
axes[0].set_xlabel("Acceleration (m/s2)")
```

```
axes[0].set_ylabel("Frequency")
```

```
axes[0].set_title("Acceleration Distribution")
```

```
# -----
```

```
# Right Plot: Turn-Rate Distribution
```

```
# -----
```

```
axes[1].hist(
```

```
    eda_data["turn_rate_radps"],
```

```
    bins=80,                    # Same resolution for consistency across plots.
```

```
    color="#E45756",           # Muted red for contrast with acceleration plot.
```

```
    edgecolor="black"
```

```
)
```

```
axes[1].set_xlabel("Turn rate (rad/s)")
```

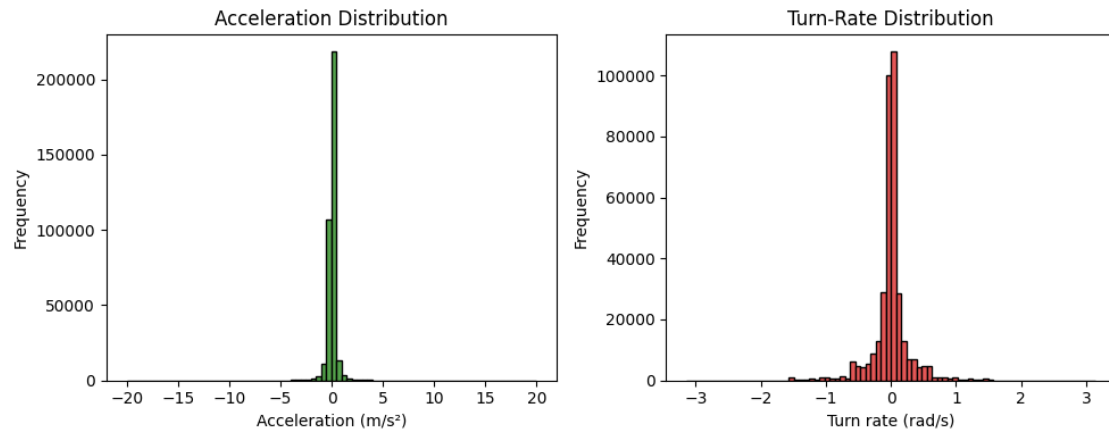
```
axes[1].set_ylabel("Frequency")
```

```
axes[1].set_title("Turn-Rate Distribution")
```

```
# Ensure spacing is clean and labels are not clipped.
```

```
plt.tight_layout()
```

```
plt.show()
```



[]: